

JAVA QB UNIT – 1 (2 marks)

1. Define a Java Servlet

1. A Java Servlet is a Java class that runs on a server. It is used to handle client requests and generate responses.
 2. It follows the request-response model. The browser sends a request and the servlet sends back a response.
 3. Servlets are mainly used to create dynamic web applications. They extend the capabilities of web servers.
 4. They use packages like `javax.servlet` and `javax.servlet.http`. All servlets must implement the Servlet interface.
-

2. Name the four tiers of J2EE architecture

1. Client Tier is the presentation layer. It includes web browsers or mobile apps that send requests.
 2. Web Tier processes the client request. It uses Servlets and JSP to generate dynamic responses.
 3. Business Tier contains business logic. It handles validations and application rules.
 4. EIS Tier is the data layer. It stores and manages data using databases.
-

3. What is the role of J2EE architecture in web applications?

1. J2EE provides a platform for building enterprise web applications. It supports distributed and multi-tier applications.
 2. It separates the application into different layers. This makes the system organized and easy to manage.
 3. It provides services like security and database connectivity. It also supports transaction management.
 4. It is used in banking, e-commerce, and ERP systems. It helps build large and secure applications.
-

4. Differentiate between Web Server and Web Container

1. A Web Server handles HTTP requests from clients. It mainly serves static content like HTML and images.
 2. A Web Container executes servlets and JSP. It manages the lifecycle of servlets.
 3. A Web Server cannot run Java servlets directly. A Web Container can process dynamic content.
 4. An example of a Web Server is Apache HTTP Server. An example of a Web Container is Apache Tomcat.
-

5. List the phases of the Servlet Life Cycle

1. The first phase is `init()`. It is called only once when the servlet is created.
2. The second phase is `service()`. It is called every time a request is received.
3. The `service()` method processes the request. It generates and sends a response to the client.
4. The final phase is `destroy()`. It is called once when the servlet is removed or the server stops.

6. Name any two HTTP methods used in servlets.

1. GET is an HTTP method used to retrieve data from the server. It sends data through the URL.
 2. POST is an HTTP method used to send data to the server. It sends data inside the request body.
 3. GET is generally used for fetching information. It is not secure for sensitive data.
 4. POST is used for form submission like login or registration. It is more secure than GET.
-

7. What is the difference between GenericServlet and HttpServlet?

1. GenericServlet is protocol-independent. HttpServlet is HTTP protocol-specific.
 2. In GenericServlet, we override the service() method. In HttpServlet, we override doGet() or doPost() methods.
 3. GenericServlet belongs to javax.servlet package. HttpServlet belongs to javax.servlet.http package.
 4. HttpServlet is mostly used in web applications. GenericServlet is rarely used for web-based HTTP applications.
-

8. Mention the purpose of the doGet() and doPost() methods.

1. The doGet() method handles GET requests from the client. It is used to retrieve data from the server.
 2. The doPost() method handles POST requests. It is used to send data to the server.
 3. doGet() sends data through the URL. It is suitable for small amounts of data.
 4. doPost() sends data inside the request body. It is suitable for large or sensitive data.
-

9. State any two advantages of using servlets over CGI.

1. Servlets are more efficient than CGI. They use multithreading instead of creating a new process for each request.
2. Servlets provide better performance. They remain in memory after the first request.
3. Servlets are portable. They can run on any server that supports Java.
4. Servlets are more secure. They run inside the Java Virtual Machine (JVM).

10. Mention different ways of creating servlet.

1. By implementing the Servlet interface. All lifecycle methods must be defined manually.
2. By extending GenericServlet class. We need to override the service() method.
3. By extending HttpServlet class. We override doGet() and doPost() methods.
4. By using annotations like @WebServlet. It removes the need of web.xml configuration

11. Define Web Server. List its roles and limitations.

1. A Web Server is software that handles HTTP requests from clients. It sends responses back to the browser.
2. It serves static content like HTML, CSS, JavaScript, and images. It manages basic security and connections.
3. It works at the presentation level of a web application. It communicates directly with the client browser.
4. A web server cannot execute servlets or JSP by itself. It can only serve static files, so it needs a web container for dynamic content.

12. Define Web Container.

1. A Web Container is also called a Servlet Container. It is responsible for executing servlets and JSP.
 2. It manages the lifecycle of servlets. It handles initialization and destruction of servlets.
 3. It processes client requests and generates dynamic responses. It also manages sessions and security.
 4. It is part of an application server or can run standalone. Example: Apache Tomcat.
-

13. Explain different types of servlets.

1. GenericServlet is a protocol-independent servlet. It overrides the service() method.
 2. HttpServlet is HTTP protocol-specific. It overrides doGet() and doPost() methods.
 3. GenericServlet belongs to javax.servlet package. HttpServlet belongs to javax.servlet.http package.
 4. HttpServlet is mostly used in web applications. GenericServlet is used when the protocol is not limited to HTTP.
-

14. Which packages are used for writing servlets?

1. The javax.servlet package is used for basic servlet classes. It contains core interfaces and classes.
 2. The javax.servlet.http package is used for HTTP-specific servlets. It provides HttpServlet and HttpSession classes.
 3. These packages provide request and response objects. They help in handling client-server communication.
 4. They also provide classes for cookies and session management. These are essential for web applications.
-

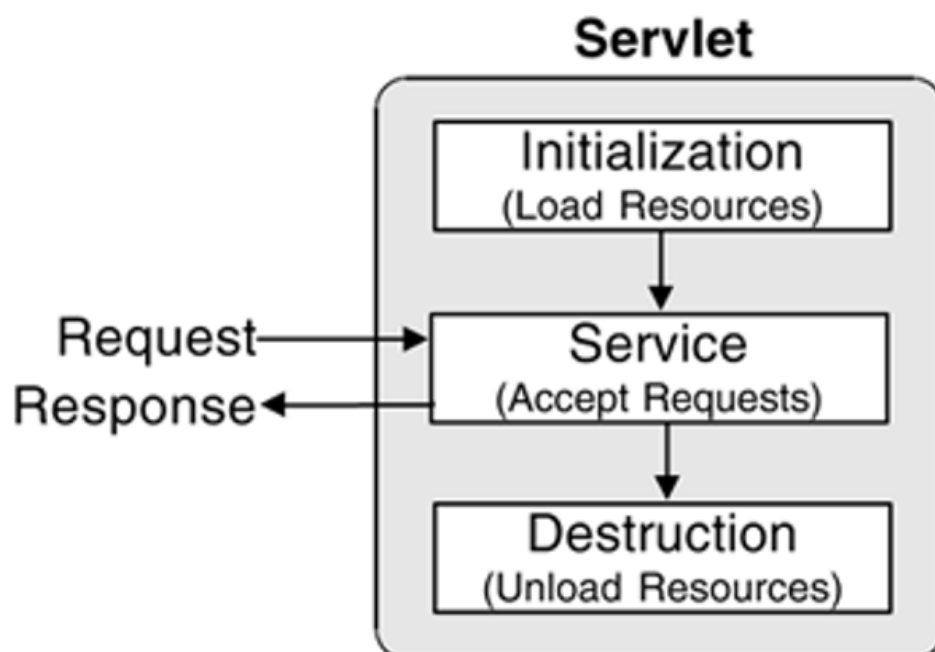
15. Explain the role of Web Tier in J2EE.

1. The Web Tier is the presentation logic layer in J2EE. It runs on the web server.
2. It receives HTTP requests from the client. It processes the request and sends a response.
3. It uses technologies like Servlets and JSP. It generates dynamic web content.
4. It forwards requests to the Business Tier for processing. It acts as a bridge between client and business logic.

(5 marks)

1. Explain the Servlet Life Cycle with a neat diagram.

1. The servlet life cycle is controlled by the web container. It defines how a servlet is created, used, and destroyed.
2. The first phase is **init()**. This method is called only once when the servlet is loaded for the first time. It is used to initialize resources like database connections.
3. The second phase is **service()**. This method is called every time a client sends a request. It processes the request and sends a response back to the client.
4. In HttpServlet, the service() method calls **doGet()** or **doPost()**. These methods handle specific HTTP requests.
5. The final phase is **destroy()**. This method is called once when the server stops or servlet is removed. It is used to release resources.



2. Explain Servlet Life Cycle with an example program.

1. The servlet life cycle has three main methods: `init()`, `service()`, and `destroy()`. These methods are managed by the web container.
2. The `init()` method runs only once. It is used to perform initialization tasks like loading drivers or setting configuration.
3. The `service()` method runs every time a request comes. In `HttpServlet`, it calls `doGet()` or `doPost()` depending on the request type.
4. The `destroy()` method runs once when the servlet is removed. It is used to free resources like database connections.

Example Program:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LifeCycleServlet extends HttpServlet {

    public void init() {
        System.out.println("Servlet Initialized");
    }

    public void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws IOException {
        PrintWriter out = response.getWriter();
        out.println("Service Method Called");
    }

    public void destroy() {
        System.out.println("Servlet Destroyed");
    }
}
```

This program shows all three life cycle methods clearly. The container calls them automatically.

3. Describe the J2EE Architecture and its components.

1. J2EE (Java Enterprise Edition) is a platform used to develop large enterprise applications. It follows a multi-tier architecture.
2. The first tier is the **Client Tier**. It includes web browsers or mobile apps that send requests to the server.
3. The second tier is the **Web Tier**. It uses Servlets and JSP to process requests and generate responses.
4. The third tier is the **Business Tier**. It contains business logic and handles validations and transactions.
5. The fourth tier is the **EIS Tier (Enterprise Information System)**. It manages databases and permanent data storage.
6. This layered structure makes applications organized and scalable. It is used in banking, e-commerce, and ERP systems.

4. Discuss the differences between a Web Server and a Web Container with examples.

1. A Web Server is software that handles HTTP requests. It mainly serves static files like HTML, CSS, and images.
2. A Web Container is responsible for executing servlets and JSP. It generates dynamic content for web applications.
3. A Web Server cannot run Java servlets directly. A Web Container can manage servlet lifecycle and session tracking.
4. Web Server works only at presentation level. Web Container provides additional services like security and request dispatching.
5. Example of Web Server is Apache HTTP Server. Example of Web Container is Apache Tomcat.
6. In simple words, a Web Server serves static content. A Web Container handles dynamic Java-based web content.

5. Explain the working of HTTP protocol and its methods in servlet communication.

1. HTTP stands for HyperText Transfer Protocol. It is used for communication between the browser and the web server.
2. HTTP works on a request–response model. The client sends a request and the server sends back a response.
3. When a user enters a URL, the browser sends an HTTP request to the server. The servlet processes the request and sends an HTTP response.
4. HTTP is a stateless protocol. This means it does not remember previous requests.
5. The GET method is used to retrieve data from the server. The data is sent through the URL and is visible in the address bar.
6. The POST method is used to send data to the server. The data is sent in the request body and is more secure than GET.
7. In servlets, `doGet()` handles GET requests. The `doPost()` method handles POST requests.

6. Compare GenericServlet and HttpServlet with suitable examples.

1. GenericServlet is a protocol-independent servlet. It can handle requests of any protocol.
2. HttpServlet is protocol-specific and works only with HTTP. It is mostly used in web applications.
3. In GenericServlet, we override the `service()` method. In HttpServlet, we override `doGet()` and `doPost()` methods.
4. GenericServlet belongs to `javax.servlet` package. HttpServlet belongs to `javax.servlet.http` package.
5. GenericServlet is rarely used for web applications. HttpServlet is widely used for handling web-based requests.

Example of GenericServlet:

```
public class MyGenericServlet extends GenericServlet {  
    public void service(ServletRequest req, ServletResponse res)  
        throws IOException {  
        res.getWriter().println("Hello from GenericServlet");  
    }  
}
```

Example of HttpServlet:

```
public class MyHttpServlet extends HttpServlet {  
    public void doGet(HttpServletRequest req, HttpServletResponse res)  
        throws IOException {  
        res.getWriter().println("Hello from HttpServlet");  
    }  
}
```

7. Write the steps involved in writing, deploying, and executing a simple servlet application.

1. First, write the servlet program by extending HttpServlet. Override doGet() or doPost() method.
2. Compile the servlet file using the servlet API library. This generates a .class file.
3. Create a web application folder structure. Place the .class file inside WEB-INF/classes folder.
4. Configure the servlet in web.xml or use @WebServlet annotation. This maps the URL to the servlet class.
5. Deploy the application on a web container like Apache Tomcat. Start the server.
6. Open a browser and type the URL of the servlet. The servlet executes and displays the output in the browser.

8. Briefly explain how to write service methods in servlet.

1. A service method is used to process client requests. It generates a response and sends it back to the client.
 2. In `GenericServlet`, we override the `service()` method. This method receives `ServletRequest` and `ServletResponse` objects.
 3. In `HttpServlet`, we usually override `doGet()` or `doPost()` instead of `service()`. These methods are automatically called by the `service()` method.
 4. The first step in writing a service method is to set the response content type. This is done using `response.setContentType()`.
 5. Next, we get a `PrintWriter` object from the response. This is used to send output to the browser.
 6. Then we extract parameters from the request object if needed. Finally, we write HTML content using the `PrintWriter`.
-

9. Explain how client requests and server responses are handled using `doGet()` and `doPost()` methods.

1. When a client sends a request, the web container receives it. The container checks the HTTP method type.
 2. If the request type is GET, the container calls the `doGet()` method. If the request type is POST, it calls the `doPost()` method.
 3. The `doGet()` method is used to retrieve data from the server. The data is passed through the URL.
 4. The `doPost()` method is used to send data to the server. The data is sent inside the request body.
 5. Inside these methods, we use `HttpServletRequest` to get user input. We use `HttpServletResponse` to send output back to the browser.
 6. The servlet processes the data and generates an HTML response. The browser then displays the response to the user.
-

10. Explain init(), service(), and destroy() methods with syntax.

1. The init() method is called only once when the servlet is created. It is used to perform initialization tasks.

Syntax:

```
public void init() throws ServletException {  
    // initialization code  
}
```

2. The service() method is called for every client request. It processes the request and generates a response.

Syntax:

```
public void service(ServletRequest request,  
    ServletResponse response)  
    throws ServletException, IOException {  
    // request processing code  
}
```

3. In HttpServlet, service() calls doGet() or doPost(). These methods handle HTTP requests.
4. The destroy() method is called once before the servlet is removed. It is used to release resources.

Syntax:

```
public void destroy() {  
    // cleanup code  
}
```

11. Write a servlet to display current date and time.

1. To display current date and time, we extend HttpServlet. We override the doGet() method.
2. Inside doGet(), we set the content type to text/html. Then we get the PrintWriter object.
3. We create a Date object from java.util package. This gives the current date and time.
4. Finally, we print the date and time using the PrintWriter. The browser displays the output.

Example Program:

```
import java.io.*;
import java.util.Date;
import javax.servlet.*;
import javax.servlet.http.*;

public class DateServlet extends HttpServlet {

    public void doGet(HttpServletRequest request,
                      HttpServletResponse response)
        throws IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<h2>Current Date & Time:</h2>");
        out.println(new Date());
    }
}
```

This servlet displays the current server date and time in the browser.